

Kotlin Ktor Backend — Complete Guide (Kotlin Theme)

Tech Stack: Kotlin · Ktor · MySQL · Exposed ORM · JWT · HikariCP

1. Project Setup

- Created a new Kotlin Ktor project using Gradle.
- Added necessary plugins: - kotlin("jvm") - kotlin("plugin.serialization") - ktor plugin
- Installed dependencies for: - Ktor Server - Exposed ORM - MySQL JDBC - HikariCP - Kotlin Serialization

Folder Structure (Architecture)

```
src/
├── main/
│   ├── kotlin/
│   │   ├── com/
│   │   │   ├── sam/
│   │   │   │   ├── library/
│   │   │   │   │   ├── Application.kt
│   │   │   │   │   ├── config/
│   │   │   │   │   │   ├── DatabaseFactory.kt
│   │   │   │   │   │   ├── JwtConfig.kt
│   │   │   │   │   ├── model/
│   │   │   │   │   │   ├── Users.kt
│   │   │   │   │   │   ├── Books.kt
│   │   │   │   │   ├── api/
│   │   │   │   │   │   ├── dto/
│   │   │   │   │   │   │   ├── AuthRequest.kt
│   │   │   │   │   │   │   ├── AuthResponse.kt
│   │   │   │   │   │   │   ├── BookCreateRequest.kt
│   │   │   │   │   │   │   ├── BookUpdateRequest.kt
│   │   │   │   │   │   │   ├── BookResponse.kt
│   │   │   │   │   │   ├── routes/
│   │   │   │   │   │   │   ├── AuthRoutes.kt
│   │   │   │   │   │   │   ├── BookRoutes.kt
│   │   │   │   │   │   ├── repository/
│   │   │   │   │   │   │   ├── UserRepository.kt
│   │   │   │   │   │   │   ├── BookRepository.kt
│   │   │   │   │   ├── service/
│   │   │   │   │   │   ├── AuthService.kt
│   │   ├── resources/
│   │   │   ├── application.yaml
```

2. Database (MySQL) Setup

- Local MySQL installed + database 'library' created.
- application.yml configured with: - db.url - db.driver - db.user - db.password
- Added Exposed ORM + HikariCP for DB connection pooling.
- DatabaseFactory initializes DB & auto-creates tables using SchemaUtils.

3. Authentication Module

- Users table created with id, email, hashed_password.
- AuthRequest & AuthResponse DTOs implemented.
- Passwords hashed using BCrypt.
- JWT token generation done using JwtConfig.
- Register Route: POST /api/auth/register
- Login Route: POST /api/auth/login

4. Books CRUD Module

- Books.kt created with fields: id, title, author, isbn, available, createdAt.
- DTOs created: - BookCreateRequest - BookUpdateRequest - BookResponse
- BookRepository implemented with: - create() - getAll() - getById() - update() - delete()
- Routes created under: /api/books

5. Final API Endpoints (Postman Ready)

POST /api/auth/register

Body:

```
{ "email": "test@gmail.com", "password": "123456" }
```

POST /api/auth/login

Body:

```
{ "email": "test@gmail.com", "password": "123456" }
```

POST /api/books

Body:

```
{ "title": "Rich Dad Poor Dad", "author": "Robert Kiyosaki", "isbn": "12345" }
```

PUT /api/books/{id}

Body:

```
{ "title": "Updated Title", "available": false }
```

GET /api/books - Fetch All Books

GET /api/books/{id} - Fetch Single Book

DELETE /api/books/{id} - Delete Book

6. Key Takeaways

✓ Ktor is extremely fast and lightweight ✓ Ideal for Kotlin/Android developers ✓ Exposed ORM is type-safe and clean ✓ JWT integration is straightforward ✓ Perfect for microservices & scalable backends ✓ Kotlin + Ktor = A powerful full-stack combination